

SMART RECRUITMENT MATCHING PLATFORM

Member 5 - Contact Requests, Notifications and Administrator Module

Complete Step-by-Step Development Guide

Technology: .NET 8 | ASP.NET Core Web API | EF Core | SQL Server | JWT | HTML/CSS/JavaScript

Prepared for the 5-member manual development workflow
06 August 2026

1. Document Basis and Scope

Intha guide Member 5-ku backend-um frontend-um complete-a develop panna step-by-step plan. Smart Recruitment Matching Platform BRD-la employer candidate-kku contact request anuppurathu, candidate accept/decline pannurathu, application status change notifications, administrator user account management, and simple dashboard counts in scope-nu define pannirukku.

BRD out-of-scope items: real-time chat, email notifications, SMS notifications, payments, AI/ML CV analysis, video interviews, assessments, mobile application, and external job-board integration. Member 5 intha out-of-scope features-ai implement panna koodathu.

NOTE: Administrator fake-job vacancy suspension/restore original BRD-la explicit requirement illa. Athu team proposed extension. Supervisor approval iruntha mattum Optional Extension section-ai implement pannunga.

Area	BRD-supported responsibility	Member 5 output
Contact requests	Employer sends; candidate accepts or declines	Secure contact request API and UI
Notifications	In-application notifications for application status changes	Notification API, unread count and pages
Administrator	Manage users and view simple usage dashboard	User management, counts and admin UI
Match integrity	Administrator does not change match results	No score/ranking editing endpoint
Optional moderation	Not explicitly in original BRD	Suspend/restore vacancy only after approval

2. Member 5 Main Responsibilities

- Contact request entity, repository, service, controller and frontend pages.
- Application status Selected aana candidate-kku employer contact request send pannura logic.
- Candidate request-ai Pending state-la Accept or Decline pannura logic.
- Contact details Accepted aana mattum securely reveal pannurathu.
- In-application notification create, list, unread count, mark read and mark all read.
- Administrator dashboard counts: total users, job seekers, employers, vacancies and applications.
- Administrator user search/filter and enable/disable account functionality.
- Optional approved vacancy moderation: view, suspend, restore and reason tracking.
- All related HTML, CSS, vanilla JavaScript, validation, loading, empty and error states.
- Member 1-4 modules-oda integration contracts and end-to-end testing.

Backend	Frontend
Models, enums, EF configurations, DTOs, repositories, services, controllers, authorization, validation, tests	Job seeker contact requests, employer contact requests, notifications, admin dashboard, admin users, optional admin vacancies

3. Phase 1 - Git Branch and Prerequisite Check

3.1 Create Member 5 feature branch

Terminal commands

```
git checkout develop
git pull origin develop
git checkout -b feature/m5-contact-notifications-admin
git push -u origin feature/m5-contact-notifications-admin
```

3.2 Required modules before starting

- Member 1 authentication and JWT module must be merged into develop.
- Member 2 JobSeekerProfile and CV module must be available.
- Member 3 EmployerProfile and Vacancy module must be available.
- Member 4 JobApplication, ApplicationStatus.Selected and ownership queries must be available.

```
git checkout develop
git pull origin develop
git checkout feature/m5-contact-notifications-admin
```

```
git merge develop
```

```
dotnet restore
dotnet build
```

NOTE: Build errors iruntha new Member 5 code start panna munnadi existing errors-ai team-oda identify pannunga.

4. Phase 2 - Folder and File Ownership

```

Controllers/
  ContactRequestsController.cs
  NotificationsController.cs
  AdminController.cs

Models/Entities/
  ContactRequest.cs
  Notification.cs

Models/Enums/
  ContactRequestStatus.cs
  NotificationType.cs

DTOs/ContactRequests/
  CreateContactRequestDto.cs
  RespondContactRequestDto.cs
  ContactRequestResponseDto.cs
  ContactDetailsDto.cs

DTOs/Notifications/
  NotificationResponseDto.cs
  NotificationSummaryDto.cs

DTOs/Admin/
  AdminDashboardDto.cs
  AdminUserListItemDto.cs
  UpdateUserStatusDto.cs
  AdminVacancyListItemDto.cs      (optional extension)
  SuspendVacancyDto.cs          (optional extension)

Interfaces/Repositories/
  IContactRequestRepository.cs
  INotificationRepository.cs
  IAdminRepository.cs

Repositories/
  ContactRequestRepository.cs
  NotificationRepository.cs
  AdminRepository.cs

Interfaces/Services/
  IContactRequestService.cs
  INotificationService.cs
  IAdminService.cs

Services/
  ContactRequestService.cs
  NotificationService.cs
  AdminService.cs

Data/Configurations/
  ContactRequestConfiguration.cs
  NotificationConfiguration.cs

wwwroot/jobseeker/
  contact-requests.html
  notifications.html

wwwroot/employer/
  contact-requests.html
  notifications.html

wwwroot/admin/
  dashboard.html
  users.html
  vacancies.html                  (optional extension)

wwwroot/js/jobseeker/
  contact-requests.js
  notifications.js

wwwroot/js/employer/
  contact-requests.js

```

```

notifications.js

wwwroot/js/admin/
  dashboard.js
  users.js
  vacancies.js                      (optional extension)

wwwroot/css/
  contact-requests.css
  notifications.css
  admin.css

```

NOTE: Program.cs, ApplicationDbContext.cs, appsettings files, migrations and common frontend helpers shared files. Member 1 integration owner-kku pull request or GitHub issue moolama change request anuppunga.

5. Phase 3 - Enums

5.1 ContactRequestStatus

```

namespace Smart_team_project.Models.Enums;

public enum ContactRequestStatus
{
    Pending,
    Accepted,
    Declined
}

```

Status	Meaning	Allowed action
Pending	Employer request sent; candidate has not answered	Candidate can Accept or Decline
Accepted	Candidate approved contact sharing	Both participants can view approved contact details
Declined	Candidate rejected contact request	Contact details remain hidden

5.2 NotificationType

```

namespace Smart_team_project.Models.Enums;

public enum NotificationType
{
    ApplicationStatusChanged,
    ContactRequestReceived,
    ContactRequestAccepted,
    ContactRequestDeclined,
    VacancySuspended,
    VacancyRestored,
    General
}

```

NOTE: VacancySuspended and VacancyRestored optional moderation extension-ku mattum.

6. Phase 4 - Database Entities

6.1 ContactRequest entity

```

using Smart_team_project.Models.Enums;

namespace Smart_team_project.Models.Entities;

public class ContactRequest
{
    public int Id { get; set; }
    public int EmployerProfileId { get; set; }
    public int JobSeekerProfileId { get; set; }
    public int VacancyId { get; set; }

    public string? Message { get; set; }
    public ContactRequestStatus Status { get; set; }
}

```

```

    = ContactRequestStatus.Pending;

    public DateTime CreatedAt { get; set; }
    = DateTime.UtcNow;
    public DateTime? RespondedAt { get; set; }

    public EmployerProfile EmployerProfile { get; set; } = null!;
    public JobSeekerProfile JobSeekerProfile { get; set; } = null!;
    public Vacancy Vacancy { get; set; } = null!;
}

```

Field	Purpose
EmployerProfileId	Request send pannura employer profile
JobSeekerProfileId	Selected candidate profile
VacancyId	Contact request related vacancy
Message	Optional employer message, maximum 500 characters
Status	Pending, Accepted or Declined
CreatedAt	UTC request-created time
RespondedAt	Candidate response time

6.2 Notification entity

```

using Smart_team_project.Models.Enums;

namespace Smart_team_project.Models.Entities;

public class Notification
{
    public int Id { get; set; }
    public int UserId { get; set; }

    public NotificationType Type { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Message { get; set; } = string.Empty;

    public bool IsRead { get; set; }
    public DateTime CreatedAt { get; set; }
    = DateTime.UtcNow;
    public DateTime? ReadAt { get; set; }

    public User User { get; set; } = null!;
}

```

NOTE: Notification table-la user-facing safe text mattum store pannunga. Password, JWT, physical file path, stack trace or database exception details store panna koodathu.

7. Phase 5 - EF Core Configurations

7.1 ContactRequestConfiguration

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Smart_team_project.Models.Entities;

namespace Smart_team_project.Data.Configurations;

public class ContactRequestConfiguration
    : IEntityTypeConfiguration<ContactRequest>
{
    public void Configure(EntityTypeBuilder<ContactRequest> builder)
    {
        builder.ToTable("ContactRequests");
        builder.HasKey(item => item.Id);

        builder.Property(item => item.Message)
            .HasMaxLength(500);

        builder.Property(item => item.Status)
            .HasConversion<string>()
            .HasMaxLength(30)
    }
}

```

```

        .IsRequired();

builder.HasIndex(item => new
{
    item.EmployerProfileId,
    item.JobSeekerProfileId,
    item.VacancyId
}).IsUnique();

builder.HasOne(item => item.EmployerProfile)
    .WithMany()
    .HasForeignKey(item => item.EmployerProfileId)
    .OnDelete(DeleteBehavior.Restrict);

builder.HasOne(item => item.JobSeekerProfile)
    .WithMany()
    .HasForeignKey(item => item.JobSeekerProfileId)
    .OnDelete(DeleteBehavior.Restrict);

builder.HasOne(item => item.Vacancy)
    .WithMany()
    .HasForeignKey(item => item.VacancyId)
    .OnDelete(DeleteBehavior.Restrict);
}
}

```

NOTE: Unique composite index same employer + same candidate + same vacancy-ku duplicate contact request prevent pannum.

7.2 NotificationConfiguration

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Smart_team_project.Models.Entities;

namespace Smart_team_project.Data.Configurations;

public class NotificationConfiguration
    : IEntityTypeConfiguration<Notification>
{
    public void Configure(EntityTypeBuilder<Notification> builder)
    {
        builder.ToTable("Notifications");
        builder.HasKey(item => item.Id);

        builder.Property(item => item.Type)
            .HasConversion<string>()
            .HasMaxLength(40)
            .IsRequired();

        builder.Property(item => item.Title)
            .HasMaxLength(150)
            .IsRequired();

        builder.Property(item => item.Message)
            .HasMaxLength(1000)
            .IsRequired();

        builder.HasOne(item => item.User)
            .WithMany()
            .HasForeignKey(item => item.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        builder.HasIndex(item => new
        {
            item.UserId,
            item.IsRead,
            item.CreatedAt
        });
    }
}

```

8. Phase 6 - DTOs and Validation

8.1 Contact request DTOs

```

using System.ComponentModel.DataAnnotations;

namespace Smart_team_project.DTOs.ContactRequests;

```

```

public class CreateContactRequestDto
{
    [Range(1, int.MaxValue)]
    public int JobSeekerProfileId { get; set; }

    [Range(1, int.MaxValue)]
    public int VacancyId { get; set; }

    [StringLength(500)]
    public string? Message { get; set; }
}

using System.ComponentModel.DataAnnotations;
using Smart_team_project.Models.Enums;

namespace Smart_team_project.DTOS.ContactRequests;

public class RespondContactRequestDto
{
    [Required]
    public ContactRequestStatus Status { get; set; }
}

```

NOTE: Response API Pending value accept panna koodathu. Accepted or Declined values mattum service layer validate pannanum.

```

namespace Smart_team_project.DTOS.ContactRequests;

public class ContactRequestResponseDto
{
    public int Id { get; set; }
    public int VacancyId { get; set; }
    public string VacancyTitle { get; set; } = string.Empty;
    public string CompanyName { get; set; } = string.Empty;
    public string CandidateName { get; set; } = string.Empty;
    public string? Message { get; set; }
    public string Status { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }
    public DateTime? RespondedAt { get; set; }
}

namespace Smart_team_project.DTOS.ContactRequests;

public class ContactDetailsDto
{
    public int ContactRequestId { get; set; }
    public string VacancyTitle { get; set; } = string.Empty;

    public string? CandidateName { get; set; }
    public string? CandidateEmail { get; set; }
    public string? CandidatePhone { get; set; }

    public string? CompanyName { get; set; }
    public string? EmployerName { get; set; }
    public string? EmployerEmail { get; set; }
    public string? EmployerPhone { get; set; }
    public string? EmployerMessage { get; set; }
}

```

8.2 Notification DTOs

```

namespace Smart_team_project.DTOS.Notifications;

public class NotificationResponseDto
{
    public int Id { get; set; }
    public string Type { get; set; } = string.Empty;
    public string Title { get; set; } = string.Empty;
    public string Message { get; set; } = string.Empty;
    public bool IsRead { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? ReadAt { get; set; }
}

namespace Smart_team_project.DTOS.Notifications;

public class NotificationSummaryDto
{
    public int UnreadCount { get; set; }
    public List<NotificationResponseDto> Items { get; set; } = new();
}

```

8.3 Administrator DTOs

```
namespace Smart_team_project.DTOs.Admin;

public class AdminDashboardDto
{
    public int TotalUsers { get; set; }
    public int ActiveUsers { get; set; }
    public int DisabledUsers { get; set; }
    public int JobSeekers { get; set; }
    public int Employers { get; set; }
    public int TotalVacancies { get; set; }
    public int OpenVacancies { get; set; }
    public int TotalApplications { get; set; }
}

namespace Smart_team_project.DTOs.Admin;

public class AdminUserListItemDto
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public bool IsActive { get; set; }
    public DateTime CreatedAt { get; set; }
}

using System.ComponentModel.DataAnnotations;

namespace Smart_team_project.DTOs.Admin;

public class UpdateUserStatusDto
{
    public bool IsActive { get; set; }

    [StringLength(300)]
    public string? Reason { get; set; }
}
```

9. Phase 7 - Repository Interfaces

9.1 IContactRequestRepository

```
using Smart_team_project.Models.Entities;

namespace Smart_team_project.Interfaces.Repositories;

public interface IContactRequestRepository
{
    Task AddAsync(ContactRequest request);
    Task<ContactRequest?> GetByIdWithDetailsAsync(int id);
    Task<bool> ExistsAsync(
        int employerProfileId,
        int jobSeekerProfileId,
        int vacancyId);
    Task<List<ContactRequest>> GetSentAsync(int employerProfileId);
    Task<List<ContactRequest>> GetReceivedAsync(int jobSeekerProfileId);
    Task SaveChangesAsync();
}
```

9.2 INotificationRepository

```
using Smart_team_project.Models.Entities;

namespace Smart_team_project.Interfaces.Repositories;

public interface INotificationRepository
{
    Task AddAsync(Notification notification);
    Task<List<Notification>> GetForUserAsync(
        int userId,
        bool? unreadOnly,
        int page,
        int pageSize);
    Task<int> CountUnreadAsync(int userId);
    Task<Notification?> GetOwnedAsync(int id, int userId);
    Task MarkAllReadAsync(int userId, DateTime readAt);
    Task SaveChangesAsync();
}
```


9.3 IAdminRepository

```
using Smart_team_project.Models.Entities;
using Smart_team_project.Models.Enums;

namespace Smart_team_project.Interfaces.Repositories;

public interface IAdminRepository
{
    Task<int> CountUsersAsync();
    Task<int> CountActiveUsersAsync(bool isActive);
    Task<int> CountUsersByRoleAsync(UserRole role);
    Task<int> CountVacanciesAsync(VacancyStatus? status);
    Task<int> CountApplicationsAsync();

    Task<List<User>> GetUsersAsync(
        string? search,
        UserRole? role,
        bool? isActive,
        int page,
        int pageSize);

    Task<User?> GetUserByIdAsync(int userId);
    Task SaveChangesAsync();
}
```

10. Phase 8 - Repository Implementation Guidance

All EF Core operations async-a use pannunga. Read-only lists-ku AsNoTracking use pannunga.

Duplicate check

```
public async Task<bool> ExistsAsync(
    int employerProfileId,
    int jobSeekerProfileId,
    int vacancyId)
{
    return await _context.ContactRequests.AnyAsync(item =>
        item.EmployerProfileId == employerProfileId &&
        item.JobSeekerProfileId == jobSeekerProfileId &&
        item.VacancyId == vacancyId);
}
```

Received requests query

```
public async Task<List<ContactRequest>> GetReceivedAsync(
    int jobSeekerProfileId)
{
    return await _context.ContactRequests
        .AsNoTracking()
        .Include(item => item.EmployerProfile)
        .ThenInclude(profile => profile.User)
        .Include(item => item.JobSeekerProfile)
        .ThenInclude(profile => profile.User)
        .Include(item => item.Vacancy)
        .Where(item =>
            item.JobSeekerProfileId == jobSeekerProfileId)
        .OrderByDescending(item => item.CreatedAt)
        .ToListAsync();
}
```

Notification list query

```
public async Task<List<Notification>> GetForUserAsync(
    int userId,
    bool? unreadOnly,
    int page,
    int pageSize)
{
    var query = _context.Notifications
        .AsNoTracking()
        .Where(item => item.UserId == userId);

    if (unreadOnly == true)
    {
        query = query.Where(item => !item.IsRead);
    }

    return await query
        .OrderByDescending(item => item.CreatedAt)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .ToListAsync();
}
```

11. Phase 9 - Service Interfaces

```
public interface INotificationService
{
    Task CreateAsync(
        int userId,
        NotificationType type,
        string title,
        string message);

    Task<NotificationSummaryDto> GetMineAsync(
        int userId,
        bool? unreadOnly,
        int page,
        int pageSize);

    Task MarkReadAsync(int userId, int notificationId);
    Task MarkAllReadAsync(int userId);
}

public interface IContactRequestService
{
    Task<ContactRequestResponseDto> CreateAsync(
        int employerUserId,
        CreateContactRequestDto request);

    Task<List<ContactRequestResponseDto>> GetSentAsync(
        int employerUserId);

    Task<List<ContactRequestResponseDto>> GetReceivedAsync(
        int jobSeekerUserId);

    Task<ContactRequestResponseDto> RespondAsync(
        int jobSeekerUserId,
        int requestId,
        RespondContactRequestDto request);

    Task<ContactDetailsDto> GetContactDetailsAsync(
        int authenticatedUserId,
        int requestId);
}

public interface IAdminService
{
    Task<AdminDashboardDto> GetDashboardAsync();

    Task<List<AdminUserListItemDto>> GetUsersAsync(
        string? search,
        UserRole? role,
        bool? isActive,
        int page,
        int pageSize);

    Task UpdateUserStatusAsync(
        int administratorUserId,
        int targetUserId,
        UpdateUserStatusDto request);
}
```

12. Phase 10 - NotificationService

```
public class NotificationService : INotificationService
{
    private readonly INotificationRepository _repository;

    public NotificationService(INotificationRepository repository)
    {
        _repository = repository;
    }

    public async Task CreateAsync(
        int userId,
        NotificationType type,
        string title,
        string message)
    {
        var notification = new Notification
        {
            UserId = userId,
            Type = type,
            Title = title.Trim(),
            Message = message.Trim(),
            IsRead = false,
            CreatedAt = DateTime.UtcNow
        };
        await _repository.Create(notification);
    }
}
```

```

    };

    await _repository.AddAsync(notification);
    await _repository.SaveChangesAsync();
}

public async Task MarkReadAsync(int userId, int notificationId)
{
    var item = await _repository.GetOwnedAsync(
        notificationId,
        userId);

    if (item is null)
        throw new NotFoundException("Notification was not found.");

    if (!item.IsRead)
    {
        item.IsRead = true;
        item.ReadAt = DateTime.UtcNow;
        await _repository.SaveChangesAsync();
    }
}

public async Task MarkAllReadAsync(int userId)
{
    await _repository.MarkAllReadAsync(
        userId,
        DateTime.UtcNow);
    await _repository.SaveChangesAsync();
}
}

```

Trigger	Receiver	Example notification
Application status changed	Job seeker	Your application for Junior Developer was updated to Selected.
Contact request created	Job seeker	ABC Technologies wants to contact you regarding Junior Developer.
Contact request accepted	Employer	The candidate accepted your contact request.
Contact request declined	Employer	The candidate declined your contact request.
Vacancy suspended (optional)	Employer	Your vacancy was suspended. Reason: ...
Vacancy restored (optional)	Employer	Your vacancy was restored.

13. Phase 11 - ContactRequestService Business Logic

13.1 Create contact request validation flow

1. Authenticated user Employer role-la irukkanum.
2. EmployerProfile JWT user ID moolama load aaganum.
3. Vacancy exist aaganum and current employer own panna vacancy-a irukkanum.
4. Candidate JobApplication vacancy-ku exist aaganum.
5. Application status exactly Selected-a irukkanum.
6. JobSeekerProfileId application candidate-oda ID-oda match aaganum.
7. Same employer + candidate + vacancy duplicate contact request irukka koodathu.
8. Message trim panni maximum 500 characters enforce pannanum.
9. New status Pending; CreatedAt UTC.
10. Candidate user-kku internal notification create pannanum.

```

public async Task<ContactRequestResponseDto> CreateAsync(
    int employerUserId,
    CreateContactRequestDto request)
{
    var employer = await _employerRepository
        .GetByUserIdAsync(employerUserId)
        ?? throw new NotFoundException(
            "Employer profile was not found.");

    var vacancy = await _vacancyRepository
        .GetOwnedVacancyAsync(request.VacancyId, employer.Id)
        ?? throw new ForbiddenException(
            "You cannot contact applicants for this vacancy.");
}

```

```

var application = await _applicationRepository
    .GetByVacancyAndCandidateAsync(
        request.VacancyId,
        request.JobSeekerProfileId)
    ?? throw new NotFoundException(
        "Candidate application was not found.");

if (application.Status != ApplicationStatus.Selected)
    throw new ConflictException(
        "Contact requests are allowed only for selected candidates.");

if (await _contactRepository.ExistsAsync(
    employer.Id,
    request.JobSeekerProfileId,
    request.VacancyId))
    throw new ConflictException(
        "A contact request already exists for this candidate.");

var item = new ContactRequest
{
    EmployerProfileId = employer.Id,
    JobSeekerProfileId = request.JobSeekerProfileId,
    VacancyId = request.VacancyId,
    Message = string.IsNullOrEmpty(request.Message)
        ? null
        : request.Message.Trim(),
    Status = ContactRequestStatus.Pending,
    CreatedAt = DateTime.UtcNow
};

await _contactRepository.AddAsync(item);
await _contactRepository.SaveChangesAsync();

await _notificationService.CreateAsync(
    application.JobSeekerProfile.UserId,
    NotificationType.ContactRequestReceived,
    "New contact request",
    $"{employer.CompanyName} wants to contact you " +
    $"regarding {vacancy.JobTitle}.");

return Map(item);
}

```

13.2 Accept or decline validation flow

1. Authenticated user JobSeeker role-la irukkanum.
2. Request exist aaganum.
3. Request candidate current user profile-a irukkanum.
4. Current status Pending-a irukkanum.
5. Requested response Accepted or Declined only.
6. Status and RespondedAt update pannanum.
7. Employer user-kku correct notification create pannanum.

```

if (request.Status != ContactRequestStatus.Accepted &&
    request.Status != ContactRequestStatus.Declined)
{
    throw new BadRequestException(
        "Response must be Accepted or Declined.");
}

if (contactRequest.Status != ContactRequestStatus.Pending)
{
    throw new ConflictException(
        "This contact request has already been answered.");
}

contactRequest.Status = request.Status;
contactRequest.RespondedAt = DateTime.UtcNow;

```

13.3 Contact details visibility

Request state	Can employer view candidate contact?	Can candidate view employer contact?
Pending	No	No
Accepted	Yes	Yes
Declined	No	No

- Only the employer and candidate who are participants may call the contact-details endpoint.
- Accepted illama contact details return panna koodathu.
- Employer view: candidate name, email, phone and vacancy title.
- Job seeker view: company name, employer name/email/phone, vacancy title and employer message.
- Phone optional-a iruntha null return pannalaam; unauthorized users-ku null data return pannama 403/404 use pannunga.

14. Phase 12 - Administrator User Management

14.1 Dashboard counts

- Total users
- Active users
- Disabled users
- Total job seekers
- Total employers
- Total vacancies
- Open vacancies
- Total applications

NOTE: Administrator match score, ranking or application score breakdown-ai edit panna koodathu.

```
public async Task<AdminDashboardDto> GetDashboardAsync()
{
    return new AdminDashboardDto
    {
        TotalUsers = await _repository.CountUsersAsync(),
        ActiveUsers = await _repository.CountActiveUsersAsync(true),
        DisabledUsers = await _repository.CountActiveUsersAsync(false),
        JobSeekers = await _repository.CountUsersByRoleAsync(
            UserRole.JobSeeker),
        Employers = await _repository.CountUsersByRoleAsync(
            UserRole.Employer),
        TotalVacancies = await _repository.CountVacanciesAsync(null),
        OpenVacancies = await _repository.CountVacanciesAsync(
            VacancyStatus.Open),
        TotalApplications = await _repository.CountApplicationsAsync()
    };
}
```

14.2 Enable or disable user account

1. Only Administrator role allowed.
2. Target user exist aaganum.
3. Administrator own account-ai disable panna koodathu.
4. Reason optional but recommended; audit log entity scope-la illa-na server log use pannalaam.
5. IsActive update and UpdatedAt set pannanum.
6. Disabled users Member 1 login service-la blocked aaganum.
7. Password hash API response-la return panna koodathu.

```
public async Task UpdateUserStatusAsync(
    int administratorUserId,
    int targetUserId,
    UpdateUserStatusDto request)
{
    if (administratorUserId == targetUserId && !request.IsActive)
        throw new BadRequestException(
            "You cannot disable your own administrator account.");

    var user = await _repository.GetUserByIdAsync(targetUserId)
        ?? throw new NotFoundException("User was not found.");

    user.IsActive = request.IsActive;
    user.UpdatedAt = DateTime.UtcNow;

    await _repository.SaveChangesAsync();
}
```

15. Optional Approved Extension - Fake Job Moderation

NOTE: Intha entire section supervisor approval iruntha mattum implement pannunga. Original BRD basic administrator account management and dashboard-ai mattum explicit-a define pannirukku.

- Admin all vacancies list/search/filter panna mudiyum.
- Suspicious/fake vacancy-ai required reason-oda suspend panna mudiyum.
- Suspended vacancy job search-la appear aaga koodathu.
- Suspended vacancy new applications accept panna koodathu.
- Existing applications preserve pannanum.
- Employer suspended vacancy edit/reopen panna koodathu.
- Only administrator restore panna mudiyum.
- Closing date valid-na restore Open; expired-na Closed.
- Employer-kku internal notification create pannanum.
- Admin vacancy content, skills or scores edit panna koodathu.

```
public class SuspendVacancyDto
{
    [Required]
    [StringLength(500, MinimumLength = 10)]
    public string Reason { get; set; } = string.Empty;
}
```

Endpoint	Rule
GET /api/admin/vacancies	Search and filter all vacancies
GET /api/admin/vacancies/{id}	View employer and vacancy details
PATCH /api/admin/vacancies/{id}/suspend	Set Suspended and record reason/admin/time
PATCH /api/admin/vacancies/{id}/restore	Restore Open or Closed depending on closing date

16. Phase 13 - API Controllers

16.1 ContactRequestsController

```
[ApiController]
[Route("api/contact-requests")]
[Authorize]
public class ContactRequestsController : ControllerBase
{
    [Authorize(Roles = "Employer")]
    [HttpPost]
    public async Task<IActionResult> Create(
        CreateContactRequestDto request) { ... }

    [Authorize(Roles = "Employer")]
    [HttpGet("sent")]
    public async Task<IActionResult> GetSent() { ... }

    [Authorize(Roles = "JobSeeker")]
    [HttpGet("received")]
    public async Task<IActionResult> GetReceived() { ... }

    [Authorize(Roles = "JobSeeker")]
    [HttpPatch("{id:int}/response")]
    public async Task<IActionResult> Respond(
        int id,
        RespondContactRequestDto request) { ... }

    [Authorize(Roles = "Employer,JobSeeker")]
    [HttpGet("{id:int}/contact-details")]
    public async Task<IActionResult> GetContactDetails(int id) { ... }
}
```

Method	Endpoint	Role	Purpose
POST	/api/contact-requests	Employer	Send request to selected candidate
GET	/api/contact-requests/sent	Employer	View sent requests
GET	/api/contact-requests/received	JobSeeker	View received requests
PATCH	/api/contact-requests/{id}/response	JobSeeker	Accept or decline
GET	/api/contact-requests/{id}/contact-details	Both participant roles	View details after acceptance

16.2 NotificationsController

```
[ApiController]
[Route("api/notifications")]
[Authorize]
public class NotificationsController : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> GetMine(
        [FromQuery] bool? unreadOnly,
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 10) { ... }

    [HttpPatch("{id:int}/read")]
    public async Task<IActionResult> MarkRead(int id) { ... }

    [HttpPatch("read-all")]
    public async Task<IActionResult> MarkAllRead() { ... }
}
```

Method	Endpoint	Purpose
GET	/api/notifications	List own notifications and unread count
PATCH	/api/notifications/{id}/read	Mark one owned notification read
PATCH	/api/notifications/read-all	Mark all current user notifications read

16.3 AdminController

```
[ApiController]
[Route("api/admin")]
[Authorize(Roles = "Administrator")]
public class AdminController : ControllerBase
{
    [HttpGet("dashboard")]
    public async Task<IActionResult> GetDashboard() { ... }

    [HttpGet("users")]
    public async Task<IActionResult> GetUsers(
        string? search,
        UserRole? role,
        bool? isActive,
        int page = 1,
        int pageSize = 10) { ... }

    [HttpPatch("users/{id:int}/status")]
    public async Task<IActionResult> UpdateUserStatus(
        int id,
        UpdateUserStatusDto request) { ... }
}
```

Method	Endpoint	Purpose
GET	/api/admin/dashboard	Simple system usage counts
GET	/api/admin/users	Search/filter user accounts
PATCH	/api/admin/users/{id}/status	Enable or disable user

17. Phase 14 - Shared Integration Changes

17.1 ApplicationDbContext additions

```
public DbSet<ContactRequest> ContactRequests
    => Set<ContactRequest>();

public DbSet<Notification> Notifications
    => Set<Notification>();
```

17.2 Program.cs dependency injection

```
builder.Services.AddScoped<IContactRequestRepository,
    ContactRequestRepository>();
builder.Services.AddScoped<INotificationRepository,
```

```
NotificationRepository>();
builder.Services.AddScoped<IAdminRepository,
    AdminRepository>();

builder.Services.AddScoped<IContactRequestService,
    ContactRequestService>();
builder.Services.AddScoped<INotificationService,
    NotificationService>();
builder.Services.AddScoped<IAdminService,
    AdminService>();
```

NOTE: Member 1 shared-file owner-na GitHub issue create panni integration commit avar moolama merge pannunga.

17.3 Migration policy

```
dotnet build
dotnet ef migrations add AddContactNotificationsAndAdmin
dotnet ef database update
```

- Migration integration owner mattum create pannanum.
- Existing migrations delete panna koodathu.
- Database drop command use panna koodathu.
- Migration review panni unique index and foreign keys correct-a irukka check pannanum.

18. Phase 15 - Integration with Members 1 to 4

Member	Member 5 dependency	Integration contract
Member 1	User, JWT, roles, IsActive, common API helper	Admin account status affects login; current user ID helper
Member 2	JobSeekerProfile, candidate phone and CV/profile data	Contact details read candidate user/profile securely
Member 3	EmployerProfile, Vacancy, company phone and ownership	Contact request checks employer owns vacancy
Member 4	JobApplication, Selected status, notification calls	Only Selected candidate can receive request; status updates call INotificationService

```
// Member 4 calls this after application status update:
await _notificationService.CreateAsync(
    jobSeekerUserId,
    NotificationType.ApplicationStatusChanged,
    "Application status updated",
    $"Your application for {jobTitle} was updated " +
    $"to {newStatus}." );
```

NOTE: Circular service dependency avoid pannunga. ApplicationService -> INotificationService allowed. NotificationService ApplicationService-ai depend panna koodathu.

19. Phase 16 - Frontend Pages and User Journey

19.1 Job seeker contact requests page

- Company name
- Vacancy title
- Employer message
- Request date
- Status badge
- Pending-na Accept and Decline buttons
- Accepted-na approved contact details button/panel
- Declined-na contact details hide

```
<article class="contact-request-card">
  <h3 id="companyName"></h3>
  <p id="vacancyTitle"></p>
  <p id="requestMessage"></p>
  <span class="status-badge" id="requestStatus"></span>
  <div id="pendingActions">
```



```

    <button type="button" id="acceptButton">Accept</button>
    <button type="button" id="declineButton">Decline</button>
  </div>
  <section id="contactDetails" hidden></section>
</article>
async function respondToRequest(id, status) {
  setActionLoading(true);

  try {
    await apiRequest(
      `/contact-requests/${id}/response`,
      {
        method: "PATCH",
        body: JSON.stringify({ status })
      }
    );

    showSuccess(
      status === "Accepted"
        ? "Contact request accepted."
        : "Contact request declined."
    );

    await loadReceivedRequests();
  } catch (error) {
    showPageError(error.message);
  } finally {
    setActionLoading(false);
  }
}

```

19.2 Employer contact requests page

- Candidate name
- Vacancy title
- Request message
- Sent date
- Pending/Accepted/Declined status
- Accepted-na View Contact Details
- No resend button for duplicate request

NOTE: Send Contact Request button Member 4 employer applicant page-la application status Selected-a iruntha mattum display pannanum. Backend check compulsory.

19.3 Notifications page and header indicator

- Header notification icon unread count badge.
- All / Unread filter.
- Newest first list.
- Unread item visually clear but text label use pannunga.
- Mark one read.
- Mark all read.
- Empty state: You have no notifications.

```

async function refreshNotificationBadge() {
  const result = await apiRequest(
    "/notifications?unreadOnly=true&page=1&pageSize=10"
  );

  const badge = document.getElementById("notificationBadge");
  badge.textContent = result.unreadCount;
  badge.hidden = result.unreadCount === 0;
}

```

19.4 Administrator dashboard

- Total users card
- Active and disabled users cards
- Job seekers and employers cards
- Total/open vacancies cards
- Total applications card
- Quick link to Manage Users
- Optional approved link to Vacancy Moderation

19.5 Administrator users page

- Search by full name or email.
- Filter by role.
- Filter by active/disabled status.
- Paginated table.
- Enable/Disable action with confirmation modal.
- Current administrator own disable action hidden/blocked.
- Password/password hash never displayed.

```
async function updateUserStatus(userId, isActive) {
  const action = isActive ? "enable" : "disable";
  const confirmed = await showConfirmationModal(
    `${action} this user account?`
  );
  if (!confirmed) return;

  await apiRequest(`/admin/users/${userId}/status`, {
    method: "PATCH",
    body: JSON.stringify({ isActive })
  });

  showSuccess(
    isActive
      ? "User account enabled successfully."
      : "User account disabled successfully."
  );

  await loadUsers();
}
```

20. Phase 17 - Validation, Security and Error Handling

Area	Validation/security rule	Failure status
Create contact request	Employer owns vacancy; candidate applied and is Selected	403/404/409
Duplicate contact request	Unique combination employer + candidate + vacancy	409
Message	Optional, maximum 500 characters	400
Respond request	Correct candidate; Pending only; Accepted/Declined only	403/409/400
Contact details	Accepted and authenticated participant only	403/409
Notification read	Current user owns notification	404
Admin APIs	Administrator role only	403
Disable account	Cannot disable own admin account	400
Pagination	page >= 1; pageSize default 10 and maximum 50	400

- Request body-la employer user ID or job seeker user ID blindly trust panna koodathu.
- JWT user ID and database relations moolama ownership derive pannanum.
- Contact data endpoint normal list endpoint-la include panna koodathu.
- ProblemDetails response use pannunga.
- SQL exception, stack trace, JWT secret and server paths expose panna koodathu.
- Frontend guard security replacement illa; backend Authorize attributes compulsory.
- Buttons API call running time-la disabled-a irukkanum to prevent double submission.
- Normal feedback-ku browser alert() avoid panni shared toast/modal use pannunga.

HTTP code	Use
400	Invalid DTO, invalid response status, invalid page values
401	JWT missing or invalid
403	Wrong role or ownership failure

HTTP code	Use
404	Record not found or safe ownership masking
409	Duplicate request, non-Selected candidate, already answered request
500	Unexpected error with safe general message

21. Phase 18 - API Request and Response Examples

21.1 Send contact request

```
POST /api/contact-requests
Authorization: Bearer EMPLOYER_TOKEN
Content-Type: application/json

{
  "jobSeekerProfileId": 12,
  "vacancyId": 7,
  "message": "We would like to discuss the next stage."
}
```

201 response example

```
{
  "id": 4,
  "vacancyId": 7,
  "vacancyTitle": "Junior Software Developer",
  "companyName": "ABC Technologies",
  "candidateName": "Test Candidate",
  "message": "We would like to discuss the next stage.",
  "status": "Pending",
  "createdAt": "2026-08-06T05:00:00Z",
  "respondedAt": null
}
```

21.2 Accept request

```
PATCH /api/contact-requests/4/response
Authorization: Bearer JOBSEEKER_TOKEN
Content-Type: application/json

{
  "status": "Accepted"
}
```

21.3 Contact details after acceptance

```
GET /api/contact-requests/4/contact-details
Authorization: Bearer EMPLOYER_OR_JOBSEEKER_TOKEN
```

NOTE: Pending or Declined state-la endpoint contact details return panna koodathu.

21.4 Admin user list

```
GET /api/admin/users?search=test&role=Employer&isActive=true&page=1&pageSize=10
Authorization: Bearer ADMIN_TOKEN
```

22. Phase 19 - Smart_team_project.http Section

```
@baseUrl = https://localhost:7001
@employerToken = paste-token-here
@jobSeekerToken = paste-token-here
@adminToken = paste-token-here
@contactRequestId = 1

### Send Contact Request
POST {{baseUrl}}/api/contact-requests
Authorization: Bearer {{employerToken}}
Content-Type: application/json

{
  "jobSeekerProfileId": 12,
  "vacancyId": 7,
  "message": "We would like to discuss the next stage."
}
```

```

### Received Contact Requests
GET {{baseUrl}}/api/contact-requests/received
Authorization: Bearer {{jobSeekerToken}}

### Accept Contact Request
PATCH {{baseUrl}}/api/contact-requests/{{contactRequestId}}/response
Authorization: Bearer {{jobSeekerToken}}
Content-Type: application/json

{
  "status": "Accepted"
}

### Contact Details
GET {{baseUrl}}/api/contact-requests/{{contactRequestId}}/contact-details
Authorization: Bearer {{jobSeekerToken}}

### My Notifications
GET {{baseUrl}}/api/notifications?page=1&pageSize=10
Authorization: Bearer {{jobSeekerToken}}

### Mark All Read
PATCH {{baseUrl}}/api/notifications/read-all
Authorization: Bearer {{jobSeekerToken}}

### Admin Dashboard
GET {{baseUrl}}/api/admin/dashboard
Authorization: Bearer {{adminToken}}

### Admin Users
GET {{baseUrl}}/api/admin/users?page=1&pageSize=10
Authorization: Bearer {{adminToken}}

```

NOTE: Real JWT tokens, real passwords and personal contact details GitHub-la commit panna koodathu.

23. Phase 20 - Required Tests

23.1 Contact request tests

Test	Expected
Employer sends request to Selected applicant	201 Created
Employer sends to Applied/Shortlisted applicant	409 Conflict
Employer sends for another employer vacancy	403 or 404
Candidate never applied	404
Duplicate contact request	409
Job seeker tries to create request	403
Correct candidate accepts Pending request	200
Wrong candidate responds	403 or 404
Already Accepted request answered again	409
Pending request contact details	Blocked
Accepted participant views contact details	200
Unrelated user views contact details	403 or 404
Declined request contact details	Blocked

23.2 Notification tests

Test	Expected
Status change creates job seeker notification	One unread notification
Contact request creates candidate notification	One unread notification

Test	Expected
Candidate accepts request	Employer notified
User lists own notifications	No other user records
Mark own notification read	IsRead true and ReadAt set
Mark another user notification read	404
Mark all read	Unread count becomes zero

23.3 Administrator tests

Test	Expected
Administrator opens dashboard	200 with correct counts
Employer opens admin dashboard	403
Admin searches users	Filtered results
Admin disables user	IsActive false
Disabled user tries login	Blocked by Member 1 auth service
Admin enables user	IsActive true
Admin disables own account	400
Admin response exposes PasswordHash	Never

23.4 Optional moderation tests

Test	Expected
Admin suspends vacancy with valid reason	Status Suspended
Reason less than 10 characters	400
Non-admin suspends vacancy	403
Suspended vacancy appears in job search	No
Suspended vacancy receives new application	Blocked
Employer edits suspended vacancy	Blocked
Admin restores unexpired vacancy	Open
Admin restores expired vacancy	Closed

24. Phase 21 - Recommended Implementation Order

Day/Stage	Tasks	Output
Stage 1	Pull develop, verify dependencies, create enums/entities/configurations	Database model compiles
Stage 2	Create DTOs, repository interfaces and implementations	Data access ready
Stage 3	Implement NotificationService and APIs	Notifications working
Stage 4	Implement ContactRequestService and APIs	Contact flow working
Stage 5	Implement Admin dashboard and user management	Basic admin module working

Day/Stage	Tasks	Output
Stage 6	Create frontend pages and JavaScript	Full-stack user journey
Stage 7	Integrate Member 4 status notifications and Send Contact button	End-to-end selection flow
Stage 8	Tests, bug fixes, PR review	Merge-ready branch
Optional	Approved vacancy moderation	Suspend/restore workflow

25. Phase 22 - Git Commits and Pull Request

```

git add .
git commit -m "feat(contact): add contact request entity and configuration"

git add .
git commit -m "feat(notification): implement in-application notifications"

git add .
git commit -m "feat(contact): implement contact request workflow"

git add .
git commit -m "feat(admin): add dashboard and user management"

git add .
git commit -m "feat(ui): add contact and notification pages"

git add .
git commit -m "feat(ui): add administrator pages"

git add .
git commit -m "test(contact): add contact and admin tests"

git push origin feature/m5-contact-notifications-admin
PR Title:
[M5] Implement contact requests, notifications and administrator module

Target branch:
develop

PR description must include:
- Endpoints added
- Backend files created
- Frontend files created
- Validation and authorization rules
- Migration requirement
- Build result
- Test result
- Optional moderation clearly marked

```

26. Common Problems and Fixes

Problem	Likely cause	Fix
Contact request always returns 404	Application query missing Include/incorrect profile ID	Verify VacancyId and JobSeekerProfileId query
Duplicate requests still saved	Unique index/migration not applied	Create/apply migration and keep service duplicate check
Employer cannot receive notification	EmployerProfile.User not loaded	Include User or query employer UserId
Contact details visible too early	List DTO contains contact fields	Remove contact data from normal response DTO
Mark read changes another user notification	Repository not filtering UserId	Use GetOwnedAsync(id, userId)
Admin disable has no effect on login	AuthService ignores IsActive	Coordinate with Member 1 and block disabled users
Circular DI error	Services depend on each other both ways	Use one-way dependency and repository contracts
Enum JSON validation error	String enum converter not configured	Configure JsonStringEnumConverter

Problem	Likely cause	Fix
		consistently
Program.cs merge conflict	Multiple members edited shared file	Let integration owner combine DI changes

27. Member 5 Final Checklist

27.1 Backend checklist

- ☐ ContactRequest entity and unique composite index created.
- ☐ Notification entity and indexes created.
- ☐ Enums stored consistently as strings.
- ☐ Only selected applicants can receive contact requests.
- ☐ Employer vacancy ownership validated.
- ☐ Duplicate request returns 409.
- ☐ Only correct candidate can accept/decline.
- ☐ Only Pending requests can be answered.
- ☐ Contact details appear only after Accepted.
- ☐ Notifications are user-owned and newest first.
- ☐ Unread count and mark-read actions work.
- ☐ Admin dashboard counts are correct.
- ☐ Admin can enable/disable users.
- ☐ Admin cannot disable own account.
- ☐ Password hash never returned.
- ☐ Administrator cannot edit matching scores.
- ☐ ProblemDetails and safe errors used.
- ☐ Build and tests pass.

27.2 Frontend checklist

- ☐ Job seeker contact requests page created.
- ☐ Employer contact requests page created.
- ☐ Notification list and unread badge created.
- ☐ Admin dashboard created.
- ☐ Admin user management page created.
- ☐ Pending requests show Accept/Decline.
- ☐ Accepted requests show approved contact details.

- ☐ Declined requests hide details.
- ☐ Loading, empty, success and error states exist.
- ☐ Buttons disabled during API calls.
- ☐ Confirmation modal used for decline/disable actions.
- ☐ Status shown using text and color.
- ☐ Pages responsive and keyboard accessible.
- ☐ Role guards added.

27.3 GitHub checklist

- ☐ Feature branch used; no direct push to develop/main.
- ☐ Latest develop merged before PR.
- ☐ Small meaningful commits used.
- ☐ Shared file changes coordinated with Member 1.
- ☐ No real token, password or personal data committed.
- ☐ Migration created by integration owner.
- ☐ PR targets develop.
- ☐ Another member reviewed PR.
- ☐ API contract and README updated.

28. Definition of Done and End-to-End Scenario

1. Employer opens own vacancy applicants page.
2. Employer updates an application status to Selected.
3. Job seeker receives an internal application-status notification.
4. Employer sees Send Contact Request button for the selected candidate.
5. Employer sends one contact request with an optional message.
6. Job seeker receives an internal contact-request notification.
7. Job seeker accepts or declines the request.
8. Employer receives the response notification.
9. When accepted, both participants can view only approved contact details.
10. When pending or declined, contact details remain hidden.
11. Administrator can view dashboard counts and manage account active status.
12. No chat, email or SMS functionality is added.
13. All APIs pass role, ownership, validation and duplicate tests.
14. Frontend and backend work together after merge into develop.

NOTE: Member 5 module complete aana system-oda selection-after-contact workflow, notification experience, and basic administrator control full-a ready aagum.

Appendix A - Complete Member 5 API Contract

Method	Endpoint	Role	Main success
POST	/api/contact-requests	Employer	201 Created
GET	/api/contact-requests/sent	Employer	200 list
GET	/api/contact-requests/received	JobSeeker	200 list
PATCH	/api/contact-requests/{id}/response	JobSeeker	200 updated request
GET	/api/contact-requests/{id}/contact-details	Participant	200 after Accepted
GET	/api/notifications	Authenticated	200 list + unread count
PATCH	/api/notifications/{id}/read	Authenticated owner	204
PATCH	/api/notifications/read-all	Authenticated	204
GET	/api/admin/dashboard	Administrator	200 counts
GET	/api/admin/users	Administrator	200 paged users
PATCH	/api/admin/users/{id}/status	Administrator	204
GET	/api/admin/vacancies	Administrator - optional	200 paged vacancies
PATCH	/api/admin/vacancies/{id}/suspend	Administrator - optional	204
PATCH	/api/admin/vacancies/{id}/restore	Administrator - optional	204

Appendix B - Ownership and Shared File Matrix

File/area	Owner	Member 5 action
ContactRequestsController and related service/repository	Member 5	Create and maintain
NotificationsController and related service/repository	Member 5	Create and maintain
AdminController and admin UI	Member 5	Create and maintain
User and IsActive login check	Member 1	Consume and request integration
JobSeekerProfile contact data	Member 2	Read through secure relations
EmployerProfile/Vacancy	Member 3	Use ownership and company data
JobApplication/ApplicationStatus	Member 4	Use Selected and status notification integration
Program.cs, DbContext, migrations	Member 1/integration owner	Submit exact change request
Common API helper/navigation/CSS	Member 1	Reuse; avoid duplicate helper

END OF MEMBER 5 IMPLEMENTATION GUIDE